

Grado y Curso: Grado de Diseño y Desarrollo de Videojuegos y Entornos virtuales, 4º

Nombre del proyecto: Videojuego Multithread RPC

Año de promoción: 2021-2025

Nombre y apellidos: Andrés Matías González Ros

Fecha de entrega: 03/06/2025



UNIVERSIDAD
DE DISEÑO Y
TECNOLOGÍA



VIDEOJUEGO MULTITHREAD RPC

Andrés Matías González Ros



3 DE JUNIO DE 2025
GRADO DE DISEÑO Y DESARROLLO DE VIDEOJUEGOS Y ENTORNOS
VIRTUALES, 4º CURSO
2021-2025

Índice

1. Descripción del videojuego.....	1
1.1. Descripción general del juego (ambientación y objetivo).....	1
1.2. Mecánicas de juego (Cómo se juega).....	1
2. Diseño RPC y comunicación cliente-servidor.....	2
2.1. MOVER.....	2
2.2. INSPECCIONAR CELDA.....	2
2.3. CAVAR.....	2
2.4. USAR MAPA.....	2
2.5. COLOCAR BANDERA.....	3
2.6. COMER.....	3
2.7. USAR SONAR.....	3
2.8. SALIR DEL JUEGO.....	3
3. Implementación.....	4
3.1. Estructura del Juego.....	4
3.1.1. Clase Cell.....	4
3.1.2. Clase Position.....	5
3.1.3. Clase Map.....	5
3.1.4. Clase Player.....	7
3.1.5. Clase Game.....	10
3.1.6. Clase Client.....	13
3.1.7. Clase Server.....	16
3.1.8. Clase Lib.....	23
4. Captura de trazas de ejecución.....	29
4.1. Movimiento del jugador.....	29
4.2. Inspección de una celda no excavada.....	30
4.3. Excavación y descubrimiento de un tesoro.....	31
4.4. Uso del mapa para detectar cercanía de tesoros o trampas.....	31
4.5. Colocación de una bandera.....	32
4.6. Uso del sonar.....	32
4.7. Fin de partida por quedarse sin energía.....	33
5. Diario de desarrollo.....	33

1. Descripción del videojuego

1.1. Descripción general del juego (ambientación y objetivo)

El jugador toma el rol de un explorador de cuevas que debe encontrar 5 tesoros ocultos cavando en una cuadrícula subterránea. Sin embargo, también hay trampas escondidas, por lo que el jugador deberá ser cuidadoso en cada paso. El reto consiste en lograr el objetivo en un máximo de 30 turnos, manejando bien sus recursos y energía.

1.2. Mecánicas de juego (Cómo se juega)

Al inicio de cada turno, se muestra el número de turno, la energía restante y los tesoros encontrados.

El jugador puede escoger una acción entre varias opciones:

1. Movimiento: El jugador puede desplazarse por la cuadrícula.
2. Inspección de celda: El jugador puede inspeccionar una celda para saber si ya ha sido cavada, si no, o si ha marcado una bandera (en caso de sospechar una trampa).
3. Cavar: Puede excavar en una celda, lo que consume energía. Puede revelar un tesoro, una trampa o nada.
4. Uso de mapa: Permite ver lo que hay en un radio de 2 celdas alrededor (tesoros, trampas o ambos).
5. Banderas: Puede marcar celdas sospechosas como trampas (tipo buscaminas).
6. Recuperación de energía: La energía baja con acciones, pero puede recuperarse comiendo alimentos disponibles.
7. Uso de sonar: Revisa en línea recta (norte/sur/este/oeste) si hay algo (sin especificar si es trampa o tesoro).

Cada acción puede consumir energía o no, pero todas siempre gastan un turno.

El juego termina cuando se encuentran todos los tesoros, cuando se acaban los turnos o si el jugador decide salir.

2. Diseño RPC y comunicación cliente-servidor

En este apartado se describen todas las acciones remotas (RPC) que el cliente puede solicitar al servidor a través de paquetes DataPacket, procesadas mediante sockets UDP. Cada acción está vinculada a una mecánica del juego y cuenta con los argumentos necesarios para su ejecución. Todas las acciones se encuentran manejadas en Server.cpp, dentro de la función serverThreadFun.

2.1. MOVER

- **Nombre de la acción:** MOVE
- **Argumentos necesarios:**
 - Dirección de movimiento: dx, dy
 - Posición actual del jugador
- **Efecto:** El jugador intenta moverse en la dirección indicada. Si el movimiento es válido (dentro del mapa y sin obstáculos), se actualiza su posición y se reduce su energía.
- **Importancia:** Es la base del sistema de exploración. Permite al jugador desplazarse y buscar lugares para cavar o inspeccionar.

2.2. INSPECCIONAR CELDA

- **Nombre de la acción:** INSPECT
- **Argumentos necesarios:**
 - Posición actual del jugador
- **Efecto:** Permite conocer si una celda ya fue cavada, si está sin excavar, o si tiene una bandera colocada. No afecta energía.
- **Importancia:** Ayuda a planificar cavados sin repetir acciones, y es útil para manejar trampas.

2.3. CAVAR

- **Nombre de la acción:** DIG
- **Argumentos necesarios:**
 - Posición actual del jugador
- **Efecto:** Se cava en la celda actual. Puede revelar un tesoro, una trampa o nada. Se actualiza la energía, el estado de la celda y los contadores de tesoros encontrados.
- **Importancia:** Es la acción clave para cumplir el objetivo del juego (encontrar tesoros). También representa un riesgo al poder activar trampas.

2.4. USAR MAPA

- **Nombre de la acción:** USEMAP
- **Argumentos necesarios:**
 - Posición actual del jugador
- **Efecto:** Muestra si hay tesoros o trampas en un radio de 2 celdas alrededor del jugador.
- **Importancia:** Permite anticiparse a zonas peligrosas o valiosas. Mejora la toma de decisiones estratégicas.

2.5. COLOCAR BANDERA

- **Nombre de la acción:** PLACEFLAG
- **Argumentos necesarios:**
 - Posición actual
- **Efecto:** Marca una celda como peligrosa (sospecha de trampa). No se puede colocar si ya fue cavada.

Importancia: Permite al jugador llevar un registro personal de zonas peligrosas.

2.6. COMER

- **Nombre de la acción:** EAT
- **Argumentos necesarios:**
 - Energía actual
- **Efecto:** Recupera energía del jugador.
- **Importancia:** Fundamental para la gestión de recursos. La energía es limitada y se consume con cada acción.

2.7. USAR SONAR

- **Nombre de la acción:** SONAR
- **Argumentos necesarios:**
 - Dirección del escaneo (dir)
 - Posición actual
- **Efecto:** Informa si hay algo (tesoro o trampa) en línea recta hacia la dirección indicada.
- **Importancia:** Complementa el uso del mapa y ofrece información útil a mayor distancia, aunque menos específica.

2.8. SALIR DEL JUEGO

- **Nombre de la acción:** EXIT
- **Argumentos necesarios:**
 - Ninguno
- **Efecto:** El juego termina inmediatamente. Se detiene la ejecución del servidor para ese cliente.
- **Importancia:** Permite abandonar la partida antes de tiempo o al completarse todos los turnos.

3. Implementación

La implementación del videojuego se ha desarrollado en C++ utilizando programación orientada a objetos, sockets UDP con WinSock2 y comunicación cliente-servidor basada en RPC (Remote Procedure Call). El cliente es interactivo y opera mediante una interfaz textual en la terminal, donde el jugador puede introducir comandos y observar el estado del juego en tiempo real.

3.1. Estructura del Juego

El núcleo del juego está representado por tres componentes fundamentales:

1. **Cell** – Define las propiedades de una celda individual del mapa.
2. **Map** – Implementa el mapa como una cuadrícula de celdas y proporciona funciones para interactuar con ellas.
3. **Position** – Representa las coordenadas de cualquier entidad (por ahora, el jugador) dentro del mapa.

3.1.1. Clase Cell

Cada celda contiene información esencial para la jugabilidad:

- **hasTreasure**: indica si hay un tesoro.
- **hasTrap**: indica si hay una trampa.
- **isDug**: señala si la celda ha sido excavada.
- **hasFlag**: marca si el jugador ha puesto una bandera (para advertir de una posible trampa).

Este diseño permite que cada celda tenga un estado independiente y dinámico que cambia según las acciones del jugador.

```
enum CellInfo
{
    NOTHING, TRAP, TREASURE, FLAG
};
class Cell
{
public:
    /**
     * @brief Indicates whether the cell contains a treasure.
     */
    bool hasTreasure = false;

    /**
     * @brief Indicates whether the cell contains a trap.
     */
    bool hasTrap = false;

    /**
     * @brief True if the cell has been dug by the player.
     */
    bool isDug = false;

    /**
     * @brief True if the player has placed a flag on the cell.
     */
    bool hasFlag = false;
};
```

3.1.2. Clase Position

Estructura sencilla que encapsula un par de coordenadas **x** e **y**. Esta estructura se utiliza extensamente para definir posiciones tanto del jugador como de operaciones dentro del mapa.

```
struct Position
{
    /**
     * @brief Horizontal coordinate (x-axis).
     */
    int x;

    /**
     * @brief Vertical coordinate (y-axis).
     */
    int y;
};
```

3.1.3. Clase Map

La clase **Map** encapsula la lógica de la cuadrícula del juego y la manipulación del entorno. Sus principales características son:

- Inicialización del mapa con dimensiones variables.

```
// Constructor that initializes the map with given dimensions and number of treasures
Map::Map(int w, int h, int numTreasures) : width(w), height(h)
{
    grid.resize(height, vector<Cell>(width)); // Resize the grid to match the map size
```

- Distribución aleatoria de tesoros (usando **rand()**).

```
srand((unsigned int)time(0)); // Seed the random number generator

// Place treasures randomly on the map
while (numTreasures > 0)
{
    int x = rand() % width; // Random x coordinate
    int y = rand() % height; // Random y coordinate
    if (!grid[y][x].hasTreasure) // Check if the cell doesn't already have a treasure
    {
        grid[y][x].hasTreasure = true; // Place treasure in the cell
        numTreasures--; // Decrease the number of treasures to place
    }
}
```

- Funciones de utilidad como:
 - **getCell(x, y)**: acceso directo a una celda.

```
const Cell& Map::getCell(int x, int y) const
{
    assert(isValidPosition(x, y));
    return grid[y][x];
}
```

- **isValidPosition(x, y)**: validación de coordenadas.

```
// Checks if the given position (x, y) is within valid map bounds
bool Map::isValidPosition(int x, int y) const
{
    return x >= 0 && x < width && y >= 0 && y < height; // Ensure position is within map's width and height
}
```

- **setCell(x, y, Cell)**: reemplazo explícito de una celda.

```
// Sets the given cell at coordinates (x, y) to a new cell
void Map::setCell(int x, int y, const Cell& newCell)
{
    assert(isValidPosition(x, y));
    grid[y][x] = newCell;
}
```

- **displayMap(pos)**: impresión de una representación visual del mapa en consola.

El método **displayMap()** representa cada celda con distintos caracteres según su estado:

- **P**: jugador.
- **F**: bandera.
- *****: celda excavada.
- **.**: celda no descubierta.

Este método contribuye a la interfaz textual del juego, permitiendo al jugador visualizar el entorno en cada turno, lo cual cumple con el requisito de una interfaz CLI clara e interactiva.

```
// Displays the map with the player's position, flags, and dug cells
void Map::displayMap(const Position& playerPos) const
{
    // Loop through each row (y-axis)
    for (int y = 0; y < height; ++y)
    {
        // Loop through each column (x-axis)
        for (int x = 0; x < width; ++x)
        {
            // Display player's position as 'P'
            if (playerPos.x == x && playerPos.y == y)
            {
                cout << "P ";
            }
            // Display flag as 'F'
            else if (grid[y][x].hasFlag())
            {
                cout << "F ";
            }
            // Display dug cells as '*'
            else if (grid[y][x].isDug())
            {
                cout << "* ";
            }
            // Display undiscovered cells as '.'
            else
            {
                cout << ". ";
            }
        }
        cout << endl;
    }
}
```

3.1.4. Clase Player

La clase **Player** representa al jugador dentro del juego y gestiona su estado y acciones. Sus principales características son:

- Inicialización del jugador con posición inicial (0, 0), energía a 100 y contador de tesoros en 0.

```
Player::Player() : pos{ 0,0 }, energy(100), treasuresFound(0), info{ 0, 0 } {}
// Constructor initializing position, energy, treasures found, treasures nearby and trap nearby
```

- Gestión del estado del jugador, incluyendo:
 - Energía restante.
 - Número de tesoros encontrados.
 - Posición actual en el mapa.

Acciones disponibles para el jugador:

- **move(dx, dy, map):** mueve al jugador si la nueva posición es válida, consumiendo energía.

```
// Moves the player by a specified delta (dx, dy) if the new position is valid
bool Player::move(int dx, int dy, const Map& map)
{
    Position newPos = { pos.x + dx, pos.y + dy };
    if (map.isValidPosition(newPos.x, newPos.y))
    {
        pos = newPos;
        energy -= 5;
        return true;
    }
    return false;
}
```

- **dig(map):** cava en la celda actual; si hay un tesoro, lo recolecta. Consume más energía.

```
// Digs the cell at the player's current position, revealing treasure or not
void Player::dig(Map& map)
{
    Cell& cell = map.getCell(pos.x, pos.y); // Access the cell at the player's position
    cell.isDug = true; // Mark the cell as dug
    energy -= 10; // Decrease energy for digging
    if (cell.hasTreasure) // If the cell contains treasure, increment the treasure count
    {
        treasuresFound++;
    }
}
```

- **inspect(map):** revisa si la celda actual está excavada o marcada con una bandera.

```
// Inspect the cell at the player's current position
InspectInfo Player::inspect(const Map& map) const
{
    const Cell& cell = map.getCell(pos.x, pos.y);
    return { cell.isDug, cell.hasFlag };
}
```

- **placeFlag(map)**: coloca una bandera en la celda actual.

```
// Places a flag on the cell at the player's current position
void Player::placeFlag(Map& map)
{
    Cell& cell = map.getCell(pos.x, pos.y); // Access the cell at the player's position
    cell.hasFlag = true; // Mark the cell as flagged
}
```

- **useMap(map)**: escanea un área de radio 2 alrededor del jugador para detectar tesoros o trampas.

```
// Uses the map to check the area around the player for treasures or traps
NearbyInfo Player::useMap(const Map& map) const
{
    NearbyInfo info
    {
        false, false
    };
    int range = 2;

    for (int dx = -range; dx <= range; ++dx)
    {
        for (int dy = -range; dy <= range; ++dy)
        {
            int newX = pos.x + dx;
            int newY = pos.y + dy;
            if (map.isValidPosition(newX, newY))
            {
                const Cell& cell = map.getCell(newX, newY);
                if (cell.hasTreasure)
                {
                    info.treasureNearby = true;
                }

                if (cell.hasTrap)
                {
                    info.trapNearby = true;
                }
            }
        }
    }

    return info;
}
```

- **useSonar(map, dir, x, y):** emite un sonar en una dirección dada para detectar objetos ocultos en línea recta.

```
// Uses sonar to detect treasures or traps in a specified direction
bool Player::useSonar(Map& map, char dir, int x, int y) const
{
    bool found = false;

    // Check in the specified direction (N, S, E, W) for treasures or traps
    switch (dir)
    {
        case 'N':
            for (int i = y - 1; i >= 0; --i)
            {
                if (map.getCell(x, i).hasTreasure || map.getCell(x, i).hasTrap)
                {
                    found = true;
                    break;
                }
            }
            break;
        case 'S':
            for (int i = y + 1; i < map.getHeight(); ++i)
            {
                if (map.getCell(x, i).hasTreasure || map.getCell(x, i).hasTrap)
                {
                    found = true;
                    break;
                }
            }
            break;
    }
}
```

```
        case 'E':
            for (int i = x + 1; i < map.getWidth(); ++i)
            {
                if (map.getCell(i, y).hasTreasure || map.getCell(i, y).hasTrap)
                {
                    found = true;
                    break;
                }
            }
            break;
        case 'W':
            for (int i = x - 1; i >= 0; --i)
            {
                if (map.getCell(i, y).hasTreasure || map.getCell(i, y).hasTrap)
                {
                    found = true;
                    break;
                }
            }
            break;
        default:
            cout << "Invalid direction." << endl;
    }
    return found;
}
```

- **eat():** restaura energía (simula consumir recursos).

```
// Restores energy to the player by 20 units
void Player::eat()
{
    energy += 20; // Increase energy
}
```

- Utiliza estructuras auxiliares:
 - **NearbyInfo**: indica si hay tesoros o trampas cercanas.

```
struct NearbyInfo
{
    bool treasureNearby;
    bool trapNearby;
};
```

- **InspectInfo**: informa sobre el estado de la celda actual.

```
struct InspectInfo
{
    bool isDug;
    bool hasFlag;
};
```

Esta clase constituye el núcleo de interacción del jugador con el entorno del juego, y define cómo este se desplaza, explora y toma decisiones estratégicas.

3.1.5. Clase Game

La clase **Game** encapsula la lógica del flujo del juego, integrando el mapa, el jugador, el ciclo de turnos y la interfaz textual. Sus principales características son:

- Inicialización del juego con parámetros configurables:
 - Dimensiones del mapa.
 - Número de turnos.
 - Cantidad de tesoros a encontrar.

```
// Constructor with map size and turn limit
Game::Game(int width, int height, int turns)
: map(width, height),
  turnLimit(turns),
  currentTurn(1),
  isRunning(true),
  treasuresFound(0),
  maxTreasures(5)
{}

// Constructor with map size, number of treasures, and turn limit
Game::Game(int width, int height, int numTreasures, int limit)
: map(width, height, numTreasures),
  turnLimit(limit),
  currentTurn(1),
  isRunning(true),
  treasuresFound(0),
  maxTreasures(numTreasures)
{}

// Default constructor
Game::Game()
: map(0, 0, 0),
  turnLimit(0),
  currentTurn(1),
  isRunning(false),
  treasuresFound(0),
  maxTreasures(0)
{}
```

- Gestión del estado general del juego:
 - Turno actual y límite de turnos.
 - Conteo de tesoros encontrados.
 - Estado de ejecución (activo/inactivo).

```
int turnLimit;           ///< Maximum number of turns allowed in the game.
int currentTurn;         ///< The current turn number in the game.
bool isRunning;          ///< Flag indicating whether the game is still running.
int treasuresFound;       ///< Number of treasures the player has found so far.
int maxTreasures;        ///< Total number of treasures in the game.
```

- Funciones principales:
 - **run()**: ejecuta un turno del juego, mostrando el estado actual (energía, turnos, mapa) e invocando el menú de acciones.

```
// Main game loop that runs each turn
void Game::run()
{
    cout << "Turn: " << currentTurn << "/" << turnLimit << endl;
    cout << "Energy: " << player.getEnergy() << endl;
    cout << "Treasures found: " << treasuresFound << "/" << maxTreasures << endl;
    map.displayMap(player.getPosition());
    showMenu();
}
```

- **showMenu()**: despliega las acciones posibles que el jugador puede tomar durante su turno (moverse, cavar, usar mapa, sonar, etc.).

```
// Displays the menu of available actions
void Game::showMenu()
{
    cout << "\nActions:\n"
         << "1. Move\n"
         << "2. Inspect cell\n"
         << "3. Dig\n"
         << "4. Use map\n"
         << "5. Flag\n"
         << "6. Eat\n"
         << "7. Use sonar\n"
         << "8. Exit\n"
         << "Choose an option: ";
}
```

- **Getters** y **setters** para el estado del juego, el mapa y el jugador.

```
// Getters
int Game::getTurnLimit() const
{
    return turnLimit;
}

int Game::getCurrentTurn() const
{
    return currentTurn;
}

bool Game::getIsRunning() const
{
    return isRunning;
}

Player& Game::getPlayer()
{
    return player;
}

Map& Game::getMap()
{
    return map;
}

// Setters
void Game::setCurrentTurn(int newTurn)
{
    currentTurn = newTurn;
}

void Game::setTreasuresFound(int newVal)
{
    treasuresFound = newVal;
}
```

- Integración directa con las clases **Player** y **Map**, permitiendo el manejo centralizado del progreso y desarrollo del juego.

Esta clase actúa como el motor principal que orquesta la interacción entre el jugador y el entorno, asegurando que el juego avance según las reglas y condiciones definidas. También cumple con el requisito de tener una interfaz de línea de comandos (CLI) clara, estructurada por turnos y fácil de usar.

3.1.6. Clase Client

La clase Client encapsula la lógica de interacción del jugador con el servidor del juego a través de comunicación UDP. Sus principales características son:

- **Inicialización y configuración de red:**
 - Preparación del entorno de red mediante WinSock, creación y configuración del socket UDP, y asignación dinámica del puerto de comunicación con el servidor.

```
// Requests a new port from the server and updates the server address accordingly
int obtainNewPort(SOCKET s, sockaddr_in* serverAddr, const string& prefix)
{
    PDataPacket request = new DataPacket(); // Create an empty request packet
    PDataPacket response = new DataPacket(); // Create an empty response packet
    sendtorecvfromMsg(s, serverAddr, request, response, prefix); // Send request and receive response
    return 0;
}

// Initializes WinSock library for network communication
void initializeWinSock()
{
    WSADATA wsaData;
    int result = WSAStartup(MAKEWORD(2, 2), &wsaData);
    assert(result == NO_ERROR);
    cout << "Client: WinSock started correctly" << endl;
}

// Creates and returns a UDP socket for communication
SOCKET createSocket()
{
    SOCKET s = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
    if (s == INVALID_SOCKET)
    {
        treatErrorExit("Client: Socket creation error: ", s, -1);
    }
    return s;
}

// Configures server address struct with given IP and port
sockaddr_in configureServerAddress(const char* ip, int port)
{
    sockaddr_in serverAddr;
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(port);
    inet_pton(AF_INET, ip, &(serverAddr.sin_addr.s_addr));
    return serverAddr;
}
```

- **Gestión del ciclo de comunicación con el servidor:**
 - Envío de paquetes de datos que representan las acciones del jugador y recepción de respuestas que contienen actualizaciones del estado del juego.

```
int main(int argc, char* argv[])
{
    // Check for client ID argument
    if (argc < 2)
    {
        cout << "Usage: ./UDPCClient <id>" << endl;
        return -1;
    }

    int clientId = atoi(argv[1]);
    cout << "Client: starting..." << endl;

    // Initialize networking
    initializeWinSock();
    SOCKET s = createSocket();

    // Set server address to localhost and port 4000
    sockaddr_in serverAddr = configureServerAddress("127.0.0.1", 4000);

    string prefix = "Client:";

    // Obtain new port from server
    obtainNewPort(s, &serverAddr, prefix);
    cout << "New port obtained: " << ntohs(serverAddr.sin_port) << endl;

    // Initialize game instance
    Game game(WIDTH, HEIGHT, TURNS);
    system("CLS");

    // Run main game loop
    runGameLoop(s, serverAddr, prefix, game, clientId);

    // Cleanup and close
    shutdownClient(s);

    return 0;
}
```

- Interpretación y procesamiento de respuestas:

- Funciones especializadas para manejar distintos tipos de respuesta, tales como movimientos, excavaciones, colocación de banderas, uso de sonar y recarga de energía, actualizando la representación local del jugador, mapa y estado general.

```
// Process server response for a movement action
void processMovementResponse(Player& player, PDataPacket response)
{
    if (response->canMove)
    {
        player.setPosition(response->position);
    }
    else
    {
        cout << "Invalid movement." << endl;
    }
    player.setEnergy(response->energy);
}

// Process response for a dig preview (before digging)
void processDigPreviewResponse(PDataPacket response)
{
    if (response->isDug) cout << "Cell has already been dug." << endl;
    if (response->cellInfo == FLAG) cout << "There's a flag here." << endl;
    if (!response->isDug) cout << "Cell hasn't been dug." << endl;
}
```

```
// Process dig action and update the game/map state
void processDigResponse(Game& game, Player& player, Map& map, Cell& cell, PDataPacket response, CellInfo& cellInfo)
{
    if (!response->isDug)
    {
        cout << "Cell has already been dug." << endl;
        return;
    }

    cellInfo = response->cellInfo;
    cell.isDug = true;
    cell.hasFlag = false;
    map.setCell(player.getPosition().x, player.getPosition().y, cell);
    player.setEnergy(response->energy);

    switch (cellInfo)
    {
        case NOTHING: cout << "There's nothing here." << endl; break;
        case TREASURE:
            cout << "Treasure found!" << endl;
            game.setTreasuresFound(response->treasuresFound);
            break;
        case TRAP:
            cout << "It's a trap!" << endl;
            break;
        default:
            break;
    }
}
```

```
// Process flag action and update map if applicable
void processFlagResponse(Player& player, Map& map, Cell& cell, PDataPacket response)
{
    if (response->cellInfo == FLAG)
    {
        cell.hasFlag = true;
        map.setCell(player.getPosition().x, player.getPosition().y, cell);
        cout << "Cell (" << player.getPosition().x << ", " << player.getPosition().y << ") has been flagged." << endl;
    }
    else if (response->isDug)
    {
        cout << "Can't place a flag here, because the cell has already been dug." << endl;
    }
}

// Process energy replenishment
void processEnergyResponse(Player& player, PDataPacket response)
{
    player.setEnergy(response->energy);
    cout << "You replenished your energy. Current energy: " << player.getEnergy() << endl;
}

// Process sonar direction result
void processSonarDirectionResponse(bool sonar)
{
    cout << (sonar ? "The sonar detects something in that direction." :
        "Nothing detected in that direction.") << endl;
}
```

```
// Handles the response from the server based on the player's action
void handleServerResponse(Game& game, Player& player, Map& map, Cell& cell, PDataPacket response, int action, CellInfo&
cellInfo, bool& treasureNearby, bool& trapNearby, bool& sonar, bool& isRunning)
{
    switch (action)
    {
        case 1: processMovementResponse(player, response); break;
        case 2: processDigPreviewResponse(response); break;
        case 3: processDigResponse(game, player, map, cell, response, cellInfo); break;
        case 4: displaySonarWarning(treasureNearby, trapNearby); break;
        case 5: processFlagResponse(player, map, cell, response); break;
        case 6: processEnergyResponse(player, response); break;
        case 7: processSonarDirectionResponse(sonar); break;
        default: break;
    }

    clearScreen();

    // Update shared status flags from the server response
    treasureNearby = response->treasureNearby;
    trapNearby = response->trapNearby;
    sonar = response->sonar;
    isRunning = response->isRunning;
}
```

- **Interfaz de usuario por línea de comandos (CLI):**
 - Solicita al jugador sus acciones (movimientos, sonar, etc.), despliega advertencias, mensajes y actualizaciones de estado, y mantiene la pantalla limpia para una experiencia clara y ordenada.

```
// Gets the player's action and related input (displacement or sonar direction)
int getPlayerAction(int& x, int& y, char& dir)
{
    int action;
    cin >> action;

    if (action == 1)
    {
        // If move action, get displacement values
        cout << "x displacement: "; cin >> x;
        cout << "y displacement: "; cin >> y;
    }
    else if (action == 7)
    {
        // If sonar action, get direction
        cout << "Choose a direction to fire the sonar (N/S/E/W): ";
        cin >> dir;
        dir = toupper(dir);
    }

    return action;
}
```

```
// Pauses and clears the console screen
void clearScreen()
{
    system("pause");
    system("CLS");
}

// Displays sonar warning messages based on nearby treasures or traps
void displaySonarWarning(bool treasureNearby, bool trapNearby)
{
    if (trapNearby && treasureNearby)
    {
        cout << "Be careful! There are traps and treasures nearby!" << endl;
    }
    else if (trapNearby)
    {
        cout << "Danger! Trap nearby!" << endl;
    }
    else if (treasureNearby)
    {
        cout << "Treasure nearby! Keep looking." << endl;
    }
    else
    {
        cout << "No signal of treasures nor traps nearby." << endl;
    }
}
```

- **Ciclo principal de juego:**
 - Ejecuta un bucle continuo que gestiona el flujo de cada turno: recibe la acción del jugador, la comunica al servidor, procesa la respuesta, actualiza el estado y verifica la continuación o finalización del juego.

```
// Main game loop handling input, communication, and state updates
void runGameLoop(SOCKET s, sockaddr_in& serverAddr, const string& prefix, Game& game, int clientId)
{
    bool isRunning = true;
    bool treasureNearby = false, trapNearby = false, sonar = false;
    int treasuresFound = 0;
    CellInfo cellInfo = NOTHING;
    Player& player = game.getPlayer();

    while (isRunning)
    {
        game.run();

        int x = 0, y = 0;
        char dir = ' ';
        int action = getPlayerAction(x, y, dir);

        // Prepare request packet to send to server
        PDataPacket request = new DataPacket(clientId, static_cast<Operation>(action - 1), cellInfo,
        treasureNearby, trapNearby, sonar, dir,
        player.getEnergy(), x, y, game.getCurrentTurn(),
        game.getTurnLimit(), treasuresFound,
        game.getIsRunning(), false, player.getPosition(),
        false);

        PDataPacket response = new DataPacket();

        // Send request and receive server response
        sendtoRecvFromMsg(s, &serverAddr, request, response, prefix);

        game.setCurrentTurn(response->currentTurn);

        Map& map = game.getMap();
        Cell& cell = map.getCell(player.getPosition().x, player.getPosition().y);

        // Handle server response
        handleServerResponse(game, player, map, cell, response, action, cellInfo, treasureNearby, trapNearby, sonar,
        isRunning);
    }
}
```

- **Funciones auxiliares:**
 - Métodos para limpiar la pantalla, mostrar advertencias del sonar, manejar errores y cerrar correctamente la conexión y recursos al finalizar.

Esta clase actúa como el cliente activo que permite al jugador interactuar con el entorno multijugador a través de la red, integrando la lógica local con la información enviada y recibida del servidor, y cumpliendo con la necesidad de una interfaz interactiva y receptiva en modo consola.

3.1.7. Clase Server

La clase Server implementa la lógica central del servidor del juego multijugador, gestionando múltiples clientes simultáneamente mediante comunicación UDP y un sistema de hilos independientes. Sus principales características son:

Inicialización del entorno de red:

- Configura la biblioteca WinSock para permitir la comunicación en red.
- Crea y enlaza un socket UDP al puerto y dirección IP especificados.
- Soporta la creación dinámica de sockets adicionales para cada cliente conectado.

```
// Creates a UDP socket bound to the given IP and port (0 means OS assigns port)
// Returns INVALID_SOCKET on failure
SOCKET createBoundSocket(int port = 0, const char* ip = "127.0.0.1")
{
    SOCKET s_new = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);

    if (s_new == INVALID_SOCKET)
    {
        cerr << "Server: Failed to create socket." << endl;
        return INVALID_SOCKET;
    }

    sockaddr_in my_addr{};
    my_addr.sin_family = AF_INET;
    my_addr.sin_port = htons(port);

    if (!inet_pton(AF_INET, ip, &my_addr.sin_addr.s_addr))
    {
        cerr << "Server: Failed to convert IP string to binary." << endl;
        closesocket(s_new);
        return INVALID_SOCKET;
    }

    if (::bind(s_new, (sockaddr*)&my_addr, sizeof(my_addr)) == SOCKET_ERROR)
    {
        cerr << "Server: Failed to bind socket." << endl;
        closesocket(s_new);
        return INVALID_SOCKET;
    }

    // Optional: print assigned port if port was 0
    sockaddr_in bound_addr{};
    int addr_len = sizeof(bound_addr);

    if (getsockname(s_new, (sockaddr*)&bound_addr, &addr_len) == 0)
    {
        cout << "Server: socket bound to address: " << ip << " port: " << ntohs(bound_addr.sin_port) << endl;
    }

    return s_new;
}
```

Manejo concurrente de múltiples clientes:

- Permite hasta un número máximo de conexiones (hilos) concurrentes definidos.
- Por cada cliente que se conecta, crea un hilo dedicado que mantiene una sesión de juego aislada mediante una instancia propia de la clase Game.
- Utiliza CreateThread y WaitForMultipleObjects para la gestión y sincronización de hilos.

```
// Creates a thread to handle client communication, returns thread HANDLE or NULL on failure
HANDLE createClientThread(int i, SOCKET s_new, const string& prefix)
{
    PThreadInfo thInfo = new ThreadInfo(i, s_new, prefix);

    DWORD dwThreadId;
    HANDLE hThread = CreateThread(
        NULL,           // default security attributes
        0,              // default stack size
        threadFun,       // thread function
        (void*)thInfo,   // argument to thread function
        0,              // default creation flags
        &dwThreadId      // returns thread ID
    );

    if (hThread == NULL)
    {
        cerr << prefix << "Failed to create thread for client " << i << endl;
        delete thInfo; // Avoid memory leak
    }

    return hThread;
}
```

Comunicación bidireccional con el cliente:

- Recibe paquetes de datos (DataPacket) desde el cliente, que representan acciones solicitadas (mover, excavar, usar sonar, etc.).
- Procesa estas acciones mediante funciones específicas para cada operación y devuelve una respuesta actualizada al cliente.
- Envía datos a través de sendtoMsg y recibe mediante recvfromMsg, asegurando que cada sesión esté correctamente dirigida a su cliente respectivo.

Procesamiento de acciones del juego:

- Interpreta las operaciones enviadas por el cliente y ejecuta la lógica correspondiente en el servidor.
- Las acciones incluyen: movimiento, inspección, excavación, colocación de banderas, uso de mapa o sonar, recarga de energía y salida del juego.
- Cada hilo mantiene sincronizado el estado del jugador y del mapa mediante la instancia propia de Game, asegurando consistencia y separación entre sesiones.

```
void handleMove(PDataPacket clientPacket, Player& player, Map& map)
{
    if (player.move(clientPacket->dx, clientPacket->dy, map))
    {
        clientPacket->energy = player.getEnergy();
        clientPacket->position = player.getPosition();
        clientPacket->canMove = true;
    }

    else
    {
        clientPacket->canMove = false;
    }
}

void handleInspect(PDataPacket clientPacket, Player& player, Map& map)
{
    InspectInfo iInfo = player.inspect(map);
    clientPacket->isDug = iInfo.isDug;
    clientPacket->cellInfo = (iInfo.hasFlag ? FLAG : NOTHING);
}
```

```
void handleDig(PDataPacket clientPacket, Player& player, Map& map)
{
    if (!map.getCell(player.getPosition().x, player.getPosition().y).isDug)
    {
        player.dig(map);
        map.getCell(player.getPosition().x, player.getPosition().y).hasFlag = false;

        clientPacket->isDug = true;
        clientPacket->energy = player.getEnergy();

        if (map.getCell(player.getPosition().x, player.getPosition().y).hasTreasure)
        {
            clientPacket->cellInfo = TREASURE;
            clientPacket->treasuresFound++;
        }

        else if (map.getCell(player.getPosition().x, player.getPosition().y).hasTrap)
        {
            clientPacket->cellInfo = TRAP;
        }

        else
        {
            clientPacket->cellInfo = NOTHING;
        }
    }
}

void handleUseMap(PDataPacket clientPacket, Player& player, Map& map)
{
    NearbyInfo nInfo = player.useMap(map);
    clientPacket->treasureNearby = nInfo.treasureNearby;
    clientPacket->trapNearby = nInfo.trapNearby;
}
```

```
void handlePlaceFlag(PDataPacket clientPacket, Player& player, Map& map)
{
    if (!map.getCell(player.getPosition().x, player.getPosition().y).isDug)
    {
        player.placeFlag(map);
        clientPacket->cellInfo = FLAG;
    }

    else
    {
        clientPacket->isDug = true;
    }
}

void handleEat(PDataPacket clientPacket, Player& player)
{
    player.eat();
    clientPacket->energy = player.getEnergy();
}

void handleSonar(PDataPacket clientPacket, Player& player, Map& map)
{
    clientPacket->sonar = player.useSonar(map, clientPacket->dir, player.getPosition().x, player.getPosition().y);
}
```

Ciclo de vida del servidor:

- Mantiene activo el servidor mientras no se superen los hilos máximos definidos.
- Al finalizar la sesión de un cliente (por fin de juego, salida o error), libera los recursos asociados (sockets, estructuras y memoria).
- Finaliza limpiamente al cerrar todos los hilos, liberar sockets y desmontar la configuración de WinSock.

```
int main()
{
    constexpr int port = 4000;
    constexpr const char* ip = "127.0.0.1";
    const string prefix = "Server: ";

    cout << prefix << "starting..." << endl;

    // Initialize WinSock
    if (initializeWinSock() != 0)
    {
        return -1; // Exit if WinSock initialization failed
    }

    // Create and bind UDP socket to specified IP and port
    SOCKET s = createAndBindSocket(ip, port);
    if (s == INVALID_SOCKET)
    {
        WSACleanup(); // Clean up WinSock before exiting
        return -1;
    }

    // Allocate memory for receiving data packets
    PDataPacket packet = new DataPacket();

    HANDLE hThreadArray[MAX_THREADS];
    int threadCount = 0;

    // Main loop: receive packets and spawn threads to handle clients
    while (threadCount < MAX_THREADS)
    {
        cout << prefix << "ready to recv" << endl;

        sockaddr_in client_addr{}; // Structure to store client address
        recvfromMsg(s, &client_addr, packet, prefix); // Receive data from client

        // Create a new thread to handle the client request
        hThreadArray[threadCount] = serverFun(packet, s, &client_addr, threadCount, prefix);
        threadCount++;
    }
}
```

```
// Wait for all client threads to finish execution
WaitForMultipleObjects(MAX_THREADS, hThreadArray, TRUE, INFINITE);

// Close all thread handles
for (int i = 0; i < MAX_THREADS; i++)
{
    CloseHandle(hThreadArray[i]);
}

// Free allocated memory and cleanup resources
delete packet;
closesocket(s);
WSACleanup();

cout << prefix << "cleaning up and exiting." << endl;
return 0;
```

Funciones auxiliares destacadas:

- **initializeWinSock():** Prepara el entorno de red.

```
// Initialize WinSock library for network communication
int initializeWinSock()
{
    WSADATA wsaData;
    int result = WSASStartup(MAKEWORD(2, 2), &wsaData); // Request WinSock version 2.2
    if (result != NO_ERROR)
    {
        cerr << "Server: Failed to start WinSock. Error: " << result << endl;
        return result; // Return error code if initialization failed
    }
    cout << "Server: WinSock started correctly" << endl;
    return 0; // Success
}
```

- **createAndBindSocket():** Crea y vincula un socket a la IP/puerto.

```
// Create a UDP socket, bind it to the specified IP and port
SOCKET createAndBindSocket(const char* ip, int port)
{
    // Create socket (IPv4, UDP)
    SOCKET s = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);

    if (s == INVALID_SOCKET)
    {
        cerr << "Server: Socket creation error." << endl;
        return INVALID_SOCKET; // Return error if socket creation fails
    }

    cout << "Server: socket created" << endl;

    // Setup socket address structure
    sockaddr_in my_addr{};
    my_addr.sin_family = AF_INET;
    my_addr.sin_port = htons(port); // Convert port to network byte order

    if (!inet_pton(AF_INET, ip, &(my_addr.sin_addr.s_addr)))
    {
        cerr << "Server: Error converting IP string to binary." << endl;
        closesocket(s); // Clean up socket before returning error
        return INVALID_SOCKET;
    }

    // Bind socket to the specified IP and port
    if (::bind(s, (sockaddr*)&my_addr, sizeof(my_addr)) == SOCKET_ERROR)
    {
        cerr << "Server: Bind error." << endl;
        closesocket(s); // Clean up socket before returning error
        return INVALID_SOCKET;
    }

    cout << "Server: socket bound to address: " << ip << " port: " << port << endl;
    return s; // Return the created and bound socket
}
```

- **serverFun():** Maneja la recepción inicial de un cliente y lanza su hilo correspondiente.

```
// Main server function to create socket, send initial packet and create client thread
HANDLE serverFun(PDataPacket clientPacket, SOCKET s, sockaddr_in* client_addr, int i, string prefix)
{
    SOCKET s_new = createBoundSocket(0, "127.0.0.1");

    if (s_new == INVALID_SOCKET)
    {
        treatErrorExit(format("Server: Thread[{}] socket creation or bind error.", i), s, -1);
    }

    sendtoMsg(s_new, client_addr, clientPacket, prefix);

    HANDLE hThread = createClientThread(i, s_new, prefix);

    if (hThread == NULL)
    {
        treatErrorExit("Server: CreateThread failed", s, -1);
        ExitProcess(3);
    }

    return hThread;
}
```

- **threadFun():** Ejecuta la lógica principal por cliente en un hilo separado.

```
// Thread function that serves a client with its own game instance
DWORD WINAPI threadFun(LPVOID param)
{
    PThreadInfo thInfo = (ThreadInfo*)param;
    bool serve = true;
    PDataPacket packet = new DataPacket();

    // Create a new game instance per thread/client
    Game g(WIDTH, HEIGHT, TREASURES, TURNS);

    while (serve)
    {
        cout << "Server Thread ready to recv" << endl;

        sockaddr_in client_addr{};
        recvfromMsg(thInfo->s, &client_addr, packet, "Server:");

        DataPacket clientPacket = (DataPacket)*packet;

        // Process the client packet
        if (!serverThreadFun(&clientPacket, &g))
        {
            sendtoMsg(thInfo->s, &client_addr, &clientPacket, thInfo->prefix);
        }
        else
        {
            // Exit thread if error or client requests exit
            serve = false;
        }
    }

    // Cleanup
    // closesocket(thInfo->s); // Not needed, done in ThreadInfo destructor
    delete thInfo;
    return 0;
}
```

- **serverThreadFun():** Descompone el paquete recibido y ejecuta la operación indicada sobre el modelo del juego.

```
// Main refactored server thread function
int serverThreadFun(PDataPacket clientPacket, Game* game)
{
    cout << "Server received packet" << endl;

    Player& player = game->getPlayer();
    Map& map = game->getMap();

    // Sync player state from client packet
    player.setEnergy(clientPacket->energy);
    player.setPosition(clientPacket->position);

    switch (clientPacket->operation)
    {
        case MOVE:
        {
            handleMove(clientPacket, player, map);
            break;
        }

        case INSPECT:
        {
            handleInspect(clientPacket, player, map);
            break;
        }

        case DIG:
        {
            handleDig(clientPacket, player, map);
            break;
        }

        case USEMAP:
        {
            handleUseMap(clientPacket, player, map);
            break;
        }
    }
}
```

```
        case PLACEFLAG:
        {
            handlePlaceFlag(clientPacket, player, map);
            break;
        }

        case EAT:
        {
            handleEat(clientPacket, player);
            break;
        }

        case SONAR:
        {
            handleSonar(clientPacket, player, map);
            break;
        }

        case EXIT:
        {
            clientPacket->isRunning = false;
            break;
        }
    }

    // Advance turn number unless it's the last turn
    if (clientPacket->currentTurn != clientPacket->maxTurns)
    {
        clientPacket->currentTurn++;
    }
    else
    {
        clientPacket->isRunning = false;
    }

    // End game if player runs out of energy
    if (clientPacket->energy <= 0)
    {
        clientPacket->isRunning = false;
    }
}
```

Esta clase actúa como el **núcleo del servidor multijugador**, gestionando múltiples instancias del juego en paralelo, asegurando la correcta interpretación y respuesta a las acciones de los jugadores, y manteniendo una arquitectura robusta y escalable basada en comunicación UDP con hilos independientes.

3.1.8. Clase Lib

La librería Lib implementa funciones y estructuras auxiliares fundamentales para la comunicación de red y la serialización de datos en el juego multijugador basado en UDP. Su diseño modular permite compartir código común entre servidor y cliente, facilitando la comunicación eficiente, segura y correcta.

Inicialización y definición de constantes y tipos

- Define constantes clave para el juego (dimensiones del mapa, número de tesoros, turnos máximos).

```
#define WIDTH 10
#define HEIGHT 10
#define TREASURES 5
#define TURNS 30
```

- Enumera las operaciones que los jugadores pueden realizar (Operation).

```
enum Operation
{
    MOVE,
    INSPECT,
    DIG,
    USEMAP,
    PLACEFLAG,
    EAT,
    SONAR,
    EXIT
};
```

- Declara la estructura DataPacket, que representa el paquete principal de datos intercambiado entre cliente y servidor, incluyendo estado del juego, posición, energía, etc.

```
// Structure representing the packet of data exchanged between client and server
struct DataPacket
{
    int client_id = 0; // ID of the client sending/receiving the packet
    Operation operation = Operation::MOVE; // Current operation the client is performing

    // Movement data
    int dx = 0, dy = 0; // Movement direction (delta x, delta y)

    // Inspect operation
    bool isDug = false; // Whether a cell has been dug

    // Dig operation
    CellInfo cellInfo = CellInfo::NOTHING; // Information about the cell after digging

    // Map operation
    bool trapNearby = false; // Whether a trap is detected nearby
    bool treasureNearby = false; // Whether a treasure is detected nearby

    // Sonar operation
    bool sonar = false; // Whether sonar was used
    char dir = ' '; // Direction used by sonar (N, S, E, W, etc.)

    // Game status
    int energy = 0; // Current energy level of the client
    int currentTurn = 0; // Current turn number
    int maxTurns = 0; // Maximum allowed turns in the game
    int treasuresFound = 0; // Number of treasures found
    bool isRunning = false; // Whether the game is currently running
    bool canMove = false; // Whether the player can move
    Position position = { 0, 0 }; // Current position of the client

    // Default constructor
    DataPacket() = default;
};
```

```
// Custom constructor for quickly initializing all fields
DataPacket(int clientId,
            Operation op,
            CellInfo cellInfo,
            bool trapNearby,
            bool treasureNearby,
            bool sonar,
            char dir,
            int energy,
            int dx,
            int dy,
            int currentTurn,
            int maxTurns,
            int treasuresFound,
            bool isRunning,
            bool canMove,
            const Position& pos,
            bool isDug)
    : client_id(clientId),
      operation(op),
      dx(dx),
      dy(dy),
      isDug(isDug),
      cellInfo(cellInfo),
      trapNearby(trapNearby),
      treasureNearby(treasureNearby),
      sonar(sonar),
      dir(dir),
      energy(energy),
      currentTurn(currentTurn),
      maxTurns(maxTurns),
      treasuresFound(treasuresFound),
      isRunning(isRunning),
      canMove(canMove),
      position(pos) {}
};
```

- Estructura ThreadInfo para almacenar información relevante sobre los hilos que maneja el servidor.

```
struct ThreadInfo
{
    int thread_id = 0;
    SOCKET s = INVALID_SOCKET;
    std::string prefix;

    ThreadInfo() = default;

    ThreadInfo(int threadId, SOCKET socket, const std::string& prefix)
        : thread_id(threadId), s(socket), prefix(prefix) {}

    ~ThreadInfo() {
        closesocket(s);
    }
};

using PThreadInfo = ThreadInfo*;
```

- Implementa la serialización y deserialización JSON de todas estas estructuras, facilitando el intercambio de datos en formato JSON mediante la biblioteca nlohmann::json.

```
// JSON serialization: convert DataPacket --> json
inline void to_json(json& j, const DataPacket& p)
{
    j = json
    {
        {"client_id", p.client_id},
        {"operation", p.operation},
        {"dx", p.dx}, {"dy", p.dy},
        {"isDug", p.isDug},
        {"cellInfo", p.cellInfo},
        {"trapNearby", p.trapNearby},
        {"treasureNearby", p.treasureNearby},
        {"sonar", p.sonar},
        {"dir", p.dir},
        {"energy", p.energy},
        {"currentTurn", p.currentTurn},
        {"maxTurns", p.maxTurns},
        {"treasuresFound", p.treasuresFound},
        {"isRunning", p.isRunning},
        {"canMove", p.canMove},
        {"position", {"x", p.position.x}, {"y", p.position.y}}
    };
}
```

```
// JSON deserialization: convert json --> DataPacket
inline void from_json(const json& j, DataPacket& p)
{
    j.at("client_id").get_to(p.client_id);
    j.at("operation").get_to(p.operation);
    j.at("dx").get_to(p.dx);
    j.at("dy").get_to(p.dy);
    j.at("isDug").get_to(p.isDug);
    j.at("cellInfo").get_to(p.cellInfo);
    j.at("trapNearby").get_to(p.trapNearby);
    j.at("treasureNearby").get_to(p.treasureNearby);
    j.at("sonar").get_to(p.sonar);
    j.at("dir").get_to(p.dir);
    j.at("energy").get_to(p.energy);
    j.at("currentTurn").get_to(p.currentTurn);
    j.at("maxTurns").get_to(p.maxTurns);
    j.at("treasuresFound").get_to(p.treasuresFound);
    j.at("isRunning").get_to(p.isRunning);
    j.at("canMove").get_to(p.canMove);
    j.at("position").at("x").get_to(p.position.x);
    j.at("position").at("y").get_to(p.position.y);
}

// JSON serialization for Position
inline void to_json(json& j, const Position& p)
{
    j = json{ {"x", p.x}, {"y", p.y} };
}

// JSON deserialization for Position
inline void from_json(const json& j, Position& p)
{
    j.at("x").get_to(p.x);
    j.at("y").get_to(p.y);
}
```

```
// Enum serialization for Operation using nlohmann::json macro
NLOHMANN_JSON_SERIALIZE_ENUM(Operation,
{
    {MOVE, "MOVE"},
    {INSPECT, "INSPECT"},
    {DIG, "DIG"},
    {USEMAP, "USEMAP"},
    {PLACEFLAG, "PLACEFLAG"},
    {EAT, "EAT"},
    {SONAR, "SONAR"},
    {EXIT, "EXIT"}
});

// Enum serialization for CellInfo using nlohmann::json macro
NLOHMANN_JSON_SERIALIZE_ENUM(CellInfo,
{
    {NOTHING, "NOTHING"},
    {TRAP, "TRAP"},
    {TREASURE, "TREASURE"},
    {FLAG, "FLAG"}
});

// Alias for pointer to DataPacket
using PDataPacket = DataPacket*;
```

Comunicación UDP

- **sendtoMsg()**: Serializa un paquete DataPacket a JSON y lo envía a través de un socket UDP hacia un destinatario especificado.

```
//UDP calls
int sendtoMsg(SOCKET s, sockaddr_in* dest_addr, PDataPacket packet, std::string prefix)
{
    // Serialize the DataPacket object to JSON
    json j = *packet;

    // Convert the JSON object to a string
    std::string msg = j.dump();

    // Send the serialized JSON string as a UDP datagram
    int result = sendto(s, msg.c_str(), (int)msg.size(), 0, (SOCKADDR*)dest_addr, sizeof(SOCKADDR));

    // Ensure the send was successful
    assert(result != SOCKET_ERROR);

    return result;
}
```

- **recvfromMsg()**: Recibe un paquete UDP, lo convierte desde JSON a DataPacket, y retorna la estructura deserializada.

```
int recvfromMsg(SOCKET s, sockaddr_in* sender_addr, PDataPacket response, std::string prefix)
{
    // Create a buffer to store the incoming data
    char buffer[4096];

    // Variable to hold the size of the sender's address structure
    int fromlen = sizeof(SOCKADDR);

    // Receive data from the socket into the buffer
    int result = recvfrom(s, buffer, sizeof(buffer) - 1, 0, (SOCKADDR*)sender_addr, &fromlen);

    // Ensure the receive was successful
    assert(result != SOCKET_ERROR);

    // Null-terminate the received data to safely convert it into a string
    buffer[result] = '\0';

    // Parse the received JSON string into a JSON object
    json j = json::parse(buffer);

    // Deserialize the JSON object into a DataPacket
    *response = j.get<DataPacket>();

    return result;
}
```

- **sendtorecvfromMsg()**: Función combinada que primero envía un paquete y luego espera una respuesta, ideal para comunicaciones sincronizadas.

```
//performs sendto and then recvfrom, assuming all required WinSock2 previous calls were succesfull
int sendtorecvfromMsg(SOCKET s, sockaddr_in* dest_addr, PDataPacket packet, PDataPacket response, std::string prefix)
{
    sendtoMsg(s, dest_addr, packet, prefix);
    return recvfromMsg(s, dest_addr, response, prefix);
}
```

- **recvfromsendtoMsg()**: Recibe un mensaje y contesta inmediatamente con otro, útil para protocolos de solicitud-respuesta.

```
//performs recvfrom and then sendto, assuming all required WinSock2 previous calls were succesfull
int recvfromsendtoMsg(SOCKET s, PDataPacket response, std::string prefix)
{
    sockaddr_in sender_addr;
    recvfromMsg(s, &sender_addr, response, prefix);
    return sendtoMsg(s, &sender_addr, response, prefix);
}
```

Estas funciones abstraen la complejidad de la comunicación UDP, incluyendo el manejo de buffers, serialización y verificación de errores.

Manejo de errores

- **treatError()**: Imprime un mensaje de error junto con el código de error de WinSock, y cierra el socket asociado.

```
//will print msg with WSAGetLastError, then closesocket
void treatError(const std::string msg, SOCKET s)
{
    std::cout << msg << WSAGetLastError() << std::endl;
    closesocket(s);
}
```

- **treatErrorExit()**: Similar a **treatError()**, pero adicionalmente limpia la configuración de WinSock y termina el proceso con un código de error, garantizando una salida limpia en fallos críticos.

```
//will print msg with WSAGetLastError, then closesocket, WSACleanup and ExitProcess(error)
void treatErrorExit(const std::string msg, SOCKET s, int error)
{
    std::cout << msg << WSAGetLastError() << std::endl;
    closesocket(s);
    WSACleanup();
    ExitProcess(error);
}
```

4.1. Movimiento del jugador

```
Turn: 1/30  
Energy: 100  
Treasures found: 0/5  
P  
. . . . .  
. . . . .  
. . . . .  
. . . . .  
. . . . .  
  
Actions:  
1. Move  
2. Inspect cell  
3. Dig  
4. Use map  
5. Flag  
6. Eat  
7. Use sonar  
8. Exit  
Choose an option: 1  
x displacement: 1  
y displacement: 0  
Turn: 2/30  
Energy: 95  
Treasures found: 0/5  
P  
. . . . .  
. . . . .  
. . . . .  
. . . . .  
. . . . .  
  
Actions:  
1. Move  
2. Inspect cell  
3. Dig  
4. Use map  
5. Flag  
6. Eat  
7. Use sonar  
8. Exit  
Choose an option:
```

El jugador eligió moverse una casilla hacia la derecha (eje X +1). El servidor procesó la operación MOVE, actualizó la posición y redujo ligeramente la energía debido al movimiento. Es correcto que la energía baje al desplazarse.

32

4.7. Fin de partida por quedarse sin energía

Consola del Cliente:

```
Turn: 14/30  
Energy: 5  
Treasures found: 0/5  
  
P  
  
Actions:  
1. Move  
2. Inspect cell  
3. Dig  
4. Use map  
5. Flag  
6. Eat  
7. Use sonar  
8. Exit  
Choose an option: 3  
There's nothing here.  
Client finishing...
```

Explicación:

El jugador gastó toda su energía antes de completar el objetivo (encontrar 5 tesoros). El servidor detectó que la energía llegó a 0 ($\text{energy} \leq 0$) y terminó la partida. Este final prematuro es coherente con las reglas del juego.

5. Diario de desarrollo

Idea inicial y primeros intentos

Al comenzar el proyecto, me propuse desarrollar un videojuego de voleibol multijugador utilizando **SDL** para gráficos y **SQL** para la gestión de usuarios y partidas. La idea era ambiciosa: permitir que dos clientes jugaran simultáneamente contra cada uno.

Sin embargo, tras varios días de desarrollo, surgieron problemas técnicos serios, principalmente relacionados con la **sincronización en tiempo real** entre los dos clientes. La lógica de mantener el estado del juego coherente en ambas máquinas (por ejemplo, la posición del balón o los movimientos exactos) resultó muy difícil de implementar correctamente bajo el enfoque que había adoptado.

Cambio de rumbo

Fue entonces cuando decidí releer los requisitos del proyecto, y descubrí que no era necesario implementar varios clientes ni multijugador en tiempo real. El proyecto exigía únicamente una **comunicación funcional entre un cliente y un servidor**, lo que me permitió repensar por completo el enfoque.

Así nació la nueva idea: un juego de **exploración en una cueva**, donde el jugador, en solitario, busca tesoros mientras evita trampas, gestionando su energía en un número limitado de turnos.

Diseño y estructura del nuevo juego

La idea central era sencilla, pero su implementación fue un reto:

Problemas enfrentados y soluciones

- **Mapas desincronizados:** Al principio, el cliente y el servidor generaban mapas distintos (con tesoros en diferentes posiciones). La solución fue mover la generación del mapa exclusivamente al servidor.
- **Confusión de coordenadas:** Había errores recurrentes al interpretar las posiciones (se invertían x e y). Fue necesario revisar cuidadosamente cada acceso al mapa para estandarizar el eje correcto.
- **Acciones no reflejadas:** Cavar en una celda no actualizaba el estado del mapa, y los turnos no avanzaban. Detecté que el servidor no estaba modificando correctamente el objeto Game ni retornando el estado actualizado al cliente.
- **Condiciones de fin de partida:** El juego seguía funcionando incluso cuando la energía del jugador llegaba a 0. Esto se solucionó añadiendo una verificación explícita al final de cada acción en el servidor.
- **Errores en movimiento:** El desplazamiento se aplicaba en el eje equivocado. Tuve que rehacer la lógica de movimiento y añadir mensajes claros de advertencia al jugador.
- **Acciones especiales:** Funciones como colocar una bandera, usar sonar, o detectar trampas/tesoros no respondían correctamente. Estas acciones requerían validaciones específicas y modificaciones dentro del modelo del juego, que no estaba preparado para soportarlas inicialmente.
- **Instancias de juego compartidas:** Uno de los mayores problemas fue que todos los clientes compartían la misma instancia del juego. Esto causaba una gran cantidad de comportamientos erráticos. La solución fue **mover la creación del objeto Game dentro del hilo (thread) que atiende a cada cliente**, garantizando así que cada jugador tenga su propia partida independiente.

Refactorización y mejoras finales

Una vez que el juego estuvo funcional, dediqué tiempo a:

- **Refactorizar** el código para mejorar la legibilidad, separar responsabilidades y eliminar redundancias.
- Crear una **versión Release** en la carpeta **Binaries**, permitiendo ejecutar el juego sin necesidad de recompilar.
- Integrar **nlohmann::json**, reemplazando el uso de múltiples variables individuales por **un único paquete JSON** para cada mensaje enviado o recibido. Esto simplificó la comunicación y la depuración, y redujo el riesgo de errores de interpretación.

Documentación

Finalmente, **reescribí toda la memoria del proyecto** para que reflejara el proceso completo de desarrollo:

- Incluí **capturas de pantalla** de prácticamente todo el código relevante.
- Expliqué detalladamente los problemas y sus soluciones.
- Mostré cómo evolucionó la idea desde una propuesta inicial multijugador hacia un diseño más enfocado, estable y funcional.
- Aseguré que cada clase y módulo (Client, Server, Game, Lib) estuviera bien documentado y justificado.

Reflexión

Este proyecto me enseñó la importancia de **ajustar las ideas al contexto real y a los requisitos**, y de no tener miedo a cambiar de rumbo cuando algo no está funcionando. También reforcé habilidades clave como **depuración, comunicación en red con UDP, manejo de hilos, serialización JSON y diseño modular**.

Aunque el camino fue largo y con muchos tropiezos, el resultado final fue una aplicación robusta, funcional y completamente desarrollada por mí, lo cual me deja muy satisfecho.